Bachelor of Computer Applications (Hons. AI and ML)
Semester- III

Subject: Database Management System

Assignment 6: <u>Implementation of Data Integrity Constraints</u>

By
Name: AKASH P
Reg. No.: 2511021240008

Faculty Name: Renita Joy
Faculty Signature:_____

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106
October 2026

1. Purpose of data integrity

Constraints prevent invalid or contradictory data from being stored. They make the database enforce important rules rather than relying only on users or application code. The constraints below were defined in the table creation script of Assignment 5 for the Online Food Delivery System.

Constraint	Where used	Purpose
PRIMARY KEY	All tables; composite key in Order_Items (order_id, item_id)	Uniquely identifies each record and prevents duplicate or NULL key values.
FOREIGN KEY	Addresses, Restaurants, Menu_Items, Orders, Order_Items, Payments, Reviews	Ensures that referenced records exist in the related tables and maintains referential integrity.
UNIQUE	Customers.email and phone, Restaurant_Owners.email and phone, Categories.category_name, Delivery_Agents.phone and vehicle_number, Menu_Items(restaurant_id, name), Payments.order_id and transaction_ref, Reviews.order_id	Prevents duplicate values in columns that must contain unique information, and enforces the one-to-one rules for Payments and Reviews.
NOT NULL	Customers.name, email and phone, Addresses.address_line, city and pincode, Menu_Items.price, Orders.address_id, Payments.amount and payment_method, Reviews.rating	Ensures that essential values are always supplied. Orders.agent_id is deliberately left nullable until an agent is assigned.
CHECK	Address type, opening status, item price, agent availability, order status, total amount, quantity, unit price, payment method, payment status, paid-date rule, rating (1 to 5)	Ensures that only valid values and conditions are entered into the database.
DEFAULT	Customers.registration_date, Addresses.address_type, Restaurants.opening_status, Menu_Items.is_veg and availability, Delivery_Agents.availability_status, Orders.order_datetime, order_status and total_amount, Payments.payment_status, Reviews.review_date	Automatically assigns predefined values when no value is provided during data insertion.

2. SQL queries to verify constraints

The tests assume the FoodDelivery database from Assignment 5 and the following sample rows.

```
-- Sample rows used by the tests below --
INSERT INTO Customers VALUES (1, 'Aarav Sharma', 'aarav.sharma@example.com', '9876543210',
'2026-10-01');
INSERT INTO Addresses VALUES (1, 1, '12 MG Road', 'Bengaluru', '560001', 'Home');
INSERT INTO Restaurant_Owners VALUES (1, 'Ramesh Kumar', 'ramesh.kumar@example.com',
'9811122233');
INSERT INTO Restaurants VALUES (1, 1, 'Spice Garden', '8023456789', '45 Brigade Road,
Bengaluru', 'South Indian', 'Open');
INSERT INTO Categories VALUES (1, 'Main Course', 'Rice, curries and breads');
INSERT INTO Menu_Items VALUES (1, 1, 1, 'Masala Dosa', 'Crispy dosa with potato filling',
80.00, TRUE, TRUE);
INSERT INTO Delivery_Agents VALUES (1, 'Suresh Gowda', '9900011122', 'KA01AB1234',
'Available');
INSERT INTO Orders VALUES (1001, 1, 1, 1, 1, '2026-10-20 13:15:00', 'Delivered', 160.00);
```

```
INSERT INTO Orders VALUES (1002, 1, 1, 1, NULL, '2026-10-21 19:30:00', 'Placed', 80.00);
INSERT INTO Order_Items VALUES (1001, 1, 2, 80.00);
INSERT INTO Order_Items VALUES (1002, 1, 1, 80.00);
INSERT INTO Payments VALUES (3001, 1001, 160.00, 'UPI', 'Paid', 'TXN20261020001', '2026-10-20
13:16:00');
INSERT INTO Reviews VALUES (5001, 1001, 5, 'Tasty and on time', '2026-10-20');


-- A. Primary key: duplicate customer_id should fail --
INSERT INTO Customers (customer_id, name, email, phone)
VALUES (1, 'Duplicate Customer', 'duplicate@example.com', '9000000001');

-- B. UNIQUE: duplicate email should fail --
INSERT INTO Customers (customer_id, name, email, phone)
VALUES (99, 'Email Duplicate', 'aarav.sharma@example.com', '9000000002');

-- C. UNIQUE: second payment for the same order should fail --
INSERT INTO Payments (payment_id, order_id, amount, payment_method, payment_status,
transaction_ref, payment_datetime)
VALUES (3999, 1001, 160.00, 'Card', 'Paid', 'TXN20261020999', '2026-10-20 13:20:00');

-- D. FOREIGN KEY: nonexistent category should fail --
INSERT INTO Menu_Items (item_id, restaurant_id, category_id, name, price)
VALUES (99, 1, 999, 'Unknown Category Item', 120.00);

-- E. CHECK: invalid order status should fail --
INSERT INTO Orders (order_id, customer_id, restaurant_id, address_id, order_status)
VALUES (99, 1, 1, 1, 'Lost');

-- F. CHECK: negative price should fail --
INSERT INTO Menu_Items (item_id, restaurant_id, category_id, name, price)
VALUES (98, 1, 1, 'Negative Price Item', -10.00);

-- G. CHECK: rating outside 1 to 5 should fail --
INSERT INTO Reviews (review_id, order_id, rating, comment)
VALUES (5999, 1002, 6, 'Invalid rating');

-- H. CHECK: zero quantity should fail --
INSERT INTO Order_Items (order_id, item_id, quantity, unit_price)
VALUES (1002, 1, 0, 80.00);

-- I. NOT NULL: customer without a phone number should fail --
INSERT INTO Customers (customer_id, name, email)
VALUES (97, 'No Phone', 'nophone@example.com');

-- J. Verify foreign-key links with a query (agent may be NULL) --
SELECT o.order_id, c.name AS customer, r.name AS restaurant,
       a.city, d.name AS agent_name
FROM Orders o
JOIN Customers c ON c.customer_id = o.customer_id
JOIN Restaurants r ON r.restaurant_id = o.restaurant_id
JOIN Addresses a ON a.address_id = o.address_id
LEFT JOIN Delivery_Agents d ON d.agent_id = o.agent_id
ORDER BY o.order_id;

-- K. Verify defaults (order_status omitted -> Placed) --
INSERT INTO Orders (order_id, customer_id, restaurant_id, address_id)
VALUES (90, 1, 1, 1);
SELECT order_id, order_status, total_amount FROM Orders WHERE order_id = 90;
```

```
DELETE FROM Orders WHERE order_id = 90;
```

3. Verification results

Tests A–I are expected to fail with duplicate-key, unique-key, check-constraint, foreign-key, or NOT NULL errors, and none of them stores any data. The join in J returns the two sample orders with valid related rows; order 1002 shows a NULL agent_name because no delivery agent has been assigned yet. Test K inserts the row with order_status = 'Placed' and total_amount = 0.00 by default, then deletes the temporary row.